UPC - UB - URV

Implementation Project

REDQ: Randomized Ensemble Double Q-Learning

Reinforcement Learning

Author:

Ricard Garcia Isern

UNIVERSITAT POLITÈCNICA DE CATALUNYA BARCELONATECH

UNIVERSITAT DE BARCELONA

UNIVERSITAT ROVIRA i VIRGILI

Contents

1 Introduction

Deep Reinforcement Learning (DRL) has emerged as a powerful paradigm for solving complex, high-dimensional continuous control problems. However, its real-world deployment is severely bottlenecked by data requirements. Addressing this sample inefficiency has caused a strict trade-off between model-free and model-based approaches. While model-free methods are robust, easy to tune, and computationally lightweight, they are sample-inefficient, often requiring millions of time-consuming and expensive physical interactions to learn an effective policy. On the other hand, model-based methods achieve excellent sample efficiency by learning an internal dynamics model to simulate experience. However, they remain computationally expensive and prone to model bias, where minor inaccuracies in the learned model lead to suboptimal policies.

The **Randomized Ensembled Double Q-Learning (REDQ)** algorithm [1] bridges this trade-off. It achieves sample efficiency while preserving stability and wall-clock efficiency. REDQ incorporates a high Update-To-Data (UTD) ratio, allowing the agent to perform multiple gradient updates per environment step. To prevent the severe overestimation bias and catastrophic performance collapse that typically occur at high UTD ratios, REDQ introduces an ensemble of Q-networks paired with a randomized in-target minimization strategy.

This report details the theoretical foundations and practical implementation of REDQ. It begins by constructing a robust Soft Actor-Critic (SAC) baseline, followed by the architectural upgrades required to achieve the REDQ framework. Finally, the report presents an empirical evaluation comparing their sample efficiency and convergence stability on continuous control benchmark environments.

2 Algorithm Description

2.1 Background: The Overestimation Problem

In standard Actor-Critic architectures, the Actor network seeks to find a policy that maximizes the Q-value estimated by the Critic network. Because the Critic relies on function approximation (neural networks), its estimates contain statistical noise and approximation errors. Since the Actor systematically searches for actions that yield the highest estimated Q-values, it favors actions that happen to have positive approximation errors. In standard Q-learning, the optimizer essentially treats this positive noise as a true signal, leading to an overestimation bias during the temporal difference (TD) updates. This causes the Q-values to diverge to infinity and the policy to collapse.

2.2 Baseline: Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) is an off-policy, maximum entropy reinforcement learning algorithm. Instead of exclusively maximizing the expected return, SAC modifies the standard RL objective to maximize both the expected return and the entropy of the policy. This encourages robust exploration, prevents premature convergence to local optima and provides a smoother loss landscape.

To combat the overestimation bias, SAC utilizes a Double Q-learning trick. It maintains two separate Critic networks, Q_{θ_1} and Q_{θ_2}, and uses the minimum of the two when calculating the target Q-value. The Bellman target for the Critic in SAC is defined as:

$$y = r + \gamma(1-d)\left[\min_{i=1,2} Q_{\theta_i'}(s',a') - \alpha \log \pi_\phi(a'|s')\right]$$

Where r is the reward, γ is the discount factor, d is the terminal state flag, $Q_{\theta_i'}$ are the target networks, and α is the temperature parameter that defines the relative importance of the entropy term $\log \pi_\phi$. While SAC is highly stable, its UTD ratio is restricted to $G = 1$ (one update per environment step).

2.3 REDQ: Randomized Ensembled Double Q-Learning

REDQ builds directly upon the SAC framework but restructures the Critic architecture to enable a high Update-To-Data (UTD) ratio ($G = 20$). Running multiple backpropagation updates for every single step taken in the environment drastically improves sample efficiency by extracting maximum information from the replay buffer. However, this high update frequency worsens overestimation bias far beyond what a simple Double-Q mechanism can handle, as the two networks quickly correlate and overfit.

To solve this, REDQ replaces the standard two-critic setup with an ensemble of N independent Critic networks (typically $N = 10$). Furthermore, it employs a randomized subset minimization strategy. During each update step in the UTD loop, REDQ randomly samples a subset of size M (typically $M = 2$) from the N target critics. The target Q-value is then calculated by taking the minimum across only this random subset, rather than the entire ensemble:

$$y = r + \gamma(1-d)\left[\min_{i \in \mathscr{M}} Q_{\theta_i'}(s',a') - \alpha \log \pi_\phi(a'|s')\right]$$

Furthermore, by randomly re-drawing the subset M at every single update step, REDQ decorrelates the target values from the specific networks being updated. This prevents the networks

from overfitting to their own predictions, keeping the variance across the ensemble stable and allowing the algorithm to learn at accelerated rate without mathematical divergence.

2.4 Policy Delay

A common technique in modern Actor-Critic algorithms (such as TD3 [2]) is policy delay, where the Actor network is updated less frequently than the Critic network to reduce variance and stabilize training. As noted in Appendix F of the original REDQ paper, while policy delay is crucial for stabilizing a standard SAC algorithm operating at a high UTD ratio, its impact on REDQ is mixed.

Because the randomized ensemble mechanism already strictly controls overestimation bias and variance, introducing an artificial delay to the Actor update becomes largely redundant. In fact, the authors observed that a pure REDQ implementation (without policy delay) often achieves better performance and faster convergence in continuous control environments. Following this theoretical insight the implementation developed for this project follows the original REDQ pseudocode (see Figure 1), updating the Actor and the Critic at every step of the UTD loop.

Algorithm 1 Randomized Ensembled Double Q-learning (REDQ)

Initialize policy parameters θ, N Q-function parameters ϕ_i, $i = 1, \ldots, N$, empty replay buffer $\mathcal{D}$. Set target parameters $\phi_{\text{targ},i} \leftarrow \phi_i$, for $i = 1, 2, \ldots, N$
repeat
  Take one action $a_t \sim \pi_\theta(\cdot|s_t)$. Observe reward r_t, new state s_{t+1}.
  Add data to buffer: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$
  for G updates **do**
    Sample a mini-batch $B = \{(s, a, r, s')\}$ from $\mathcal{D}$
    Sample a set $\mathcal{M}$ of M distinct indices from $\{1, 2, \ldots, N\}$
    Compute the Q target y (same for all of the N Q-functions):

$$y = r + \gamma \left(\min_{i \in \mathcal{M}} Q_{\phi_{\text{targ},i}}\left(s', \tilde{a}'\right) - \alpha \log \pi_\theta\left(\tilde{a}' \mid s'\right) \right), \quad \tilde{a}' \sim \pi_\theta\left(\cdot \mid s'\right)$$

    for $i = 1, \ldots, N$ **do**
      Update ϕ_i with gradient descent using

$$\nabla_\phi \frac{1}{|B|} \sum_{(s,a,r,s') \in B} \left(Q_{\phi_i}(s, a) - y\right)^2$$

      Update target networks with $\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho)\phi_i$
  Update policy parameters θ with gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left(\frac{1}{N} \sum_{i=1}^{N} Q_{\phi_i}\left(s, \tilde{a}_\theta(s)\right) - \alpha \log \pi_\theta\left(\tilde{a}_\theta(s) | s\right) \right), \quad \tilde{a}_\theta(s) \sim \pi_\theta(\cdot \mid s)$$

Figure 1: Randomized Ensembled Double Q-learning (REDQ) Pseudocode.

2.5 Implementation Details

2.5.1 Unified Algorithmic Framework

The implementation of the algorithms was done in *Python* using the *PyTorch* library. The architecture was designed to be highly modular, separating the environment interaction loop, the Replay Buffer (`buffer.py`), the neural network architectures (`networks.py`), and the core agent logic (`redq.py`).

Rather than using separate scripts for the SAC and REDQ algorithms, a unified algorithm has been developed. The agent relies on an Actor network that outputs the mean and standard deviation of a Gaussian distribution to sample continuous actions, using the reparameterization trick to allow gradients to flow back from the Critics. The Critic architecture consists of an ensemble of N parallel networks and N corresponding target networks. Target networks are updated using *Polyak* averaging.

The core of the algorithm resides in the Update-To-Data (UTD) loop. For every single step taken in the environment, the algorithm executes a loop of G gradient updates. Inside this loop:

1. A random subset of size M is sampled from the N target critics without replacement.
2. The target Q-value is computed using the soft Bellman equation by taking the minimum prediction across only these M selected target networks.
3. The Mean Squared Error (MSE) loss is calculated and backpropagated for all N main Critic networks simultaneously.
4. The Actor is updated by maximizing the mean of the N critics minus the entropy term.

This unified design demonstrates the theoretical continuity between the algorithms. By setting the parameters to $N = 2$, $M = 2$, and $G = 1$, the framework replicates the standard SAC baseline. By expanding to $N = 10$, $M = 2$, and $G = 20$, it transforms into the standard REDQ algorithm.

3 Experimental Setup

3.1 Environments

To evaluate the algorithms, two continuous environments (from *Gymnasium* Library [3]) were selected:

- **BipedalWalker-v3 (Box2D):** A 2D simulation of a bipedal robot. The agent must coordinate four joints (hips and knees) to walk across a procedurally generated terrain. It requires learning a complex, rhythmic locomotion policy and is heavily penalized for falling.

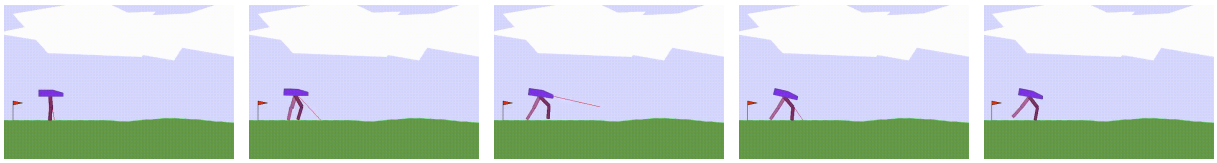


Figure 2: Sequential frames of the walking gait in BipedalWalker-v3.

- **Hopper-v5 (MuJoCo):** A 3D simulation of a single-legged robot. The goal is to make the robot hop forward as fast as possible without falling. This environment is highly unstable and prone to premature termination, making it an excellent test for algorithmic stability.

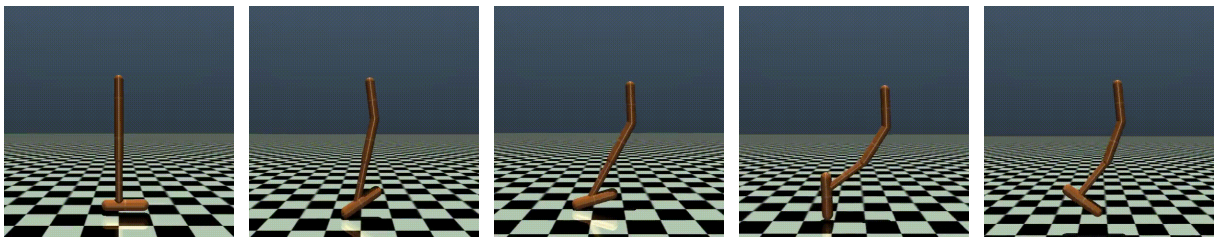


Figure 3: Sequential frames of the hopping locomotion in Hopper-v5.

3.2 Evaluation Metrics

To assess the performance and stability of the algorithms, three primary metrics were recorded throughout the training process:

- **Sample Efficiency (Average Return):** This is the primary performance indicator. Every 5,000 environment steps, the training process was paused, and the current deterministic policy (the mean of the Actor's output distribution) was evaluated over 3 independent test episodes. The average return indicates how quickly the agent learns a competent policy relative to its physical environment interactions.

- **Q-Value Bias (Overestimation Tracking):** A core objective of the REDQ architecture is to control the overestimation bias inherent in continuous Q-learning. To quantify this, the true discounted return of the evaluation episodes was tracked and compared against the internal Q-value predictions made by the Critic ensemble.
- **Computational Overhead (Wall-Clock Time):** Because high Update-To-Data ratios shift the computational cost from environment simulation to neural network backpropagation, the physical time required to complete the training loops was recorded to evaluate the practical, real-world deployment viability of the algorithm.

3.3 Protocol and Hyperparameters

Due to the high computational cost of the REDQ algorithm, the maximum number of timesteps was limited to 300,000 for Hopper-v5 and 150,000 for BipedalWalker-v3. These limits were chosen because REDQ has been shown to converge reliably within these training horizons on both benchmarks, which was also supported by the stable final performance observed in our experiments.

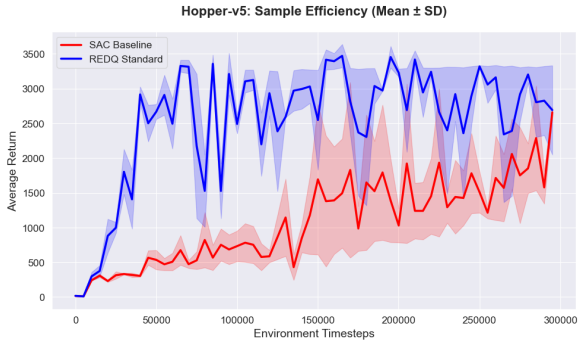
Three main experiments were designed to evaluate the algorithm's performance:

1. **Sample Efficiency Comparison:** Comparing the SAC Baseline ($G = 1, N = 2, M = 2$) against the REDQ Standard configuration ($G = 20, N = 10, M = 2$).
2. **Ablation Study (UTD Ratio G):** Comparing REDQ configurations with $G \in \{1, 10, 20\}$, keeping the ensemble fixed at N=10 and the subset at M=2, to isolate the effect of update frequency on sample efficiency.
3. **Robustness and Stability (Ensemble Size N):** Comparing the standard REDQ ensemble (N=10) against a reduced configuration (N=2) at a fixed UTD ratio of G=10, demonstrating the severe overestimation and policy collapse that occur without a sufficiently diverse ensemble.

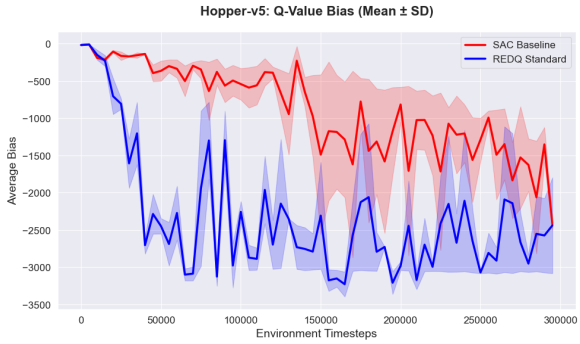
To account for the high variance inherent in reinforcement learning, each configuration was evaluated across 2 independent random seeds.

4 Results and Analysis

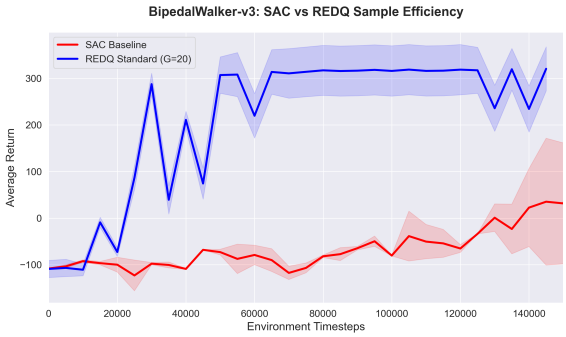
4.1 Experiment 1: Sample Efficiency Comparison



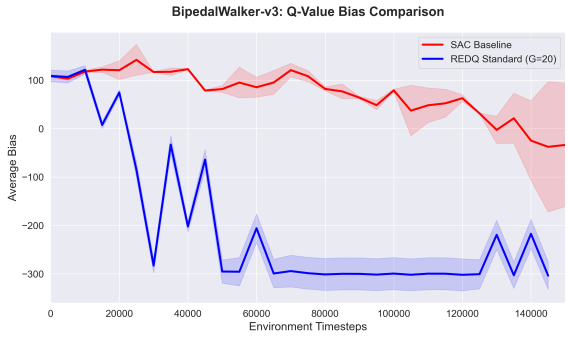
(a) Hopper-v5: Average Return



(b) Hopper-v5: Q-Value Bias



(c) BipedalWalker-v3: Average Return



(d) BipedalWalker-v3: Q-Value Bias

Figure 4: Experiment 1: Sample efficiency comparison between SAC Baseline and REDQ Standard on both environments.

Hopper-v5

The Hopper-v5 results offer a sharp illustration of REDQ's sample efficiency advantage. As shown in Figure 4 (a), REDQ achieves returns in the range of 2,800–3,000 as early as ∼35,000 environment steps, a performance level that SAC does not approach until well beyond 200,000 steps. This represents a roughly 6× improvement in sample efficiency. It is important to remark that both algorithms show significant variance throughout the training process. This fluctuation is a common characteristic of the Hopper environment. Because balancing a single-legged robot is very dificult, even a well-trained agent can occasionally make a slight mistake, lose its balance, and fall. When this happens, the episode terminates early, causing a sudden drop in the average return. However, this natural instability of the environment affects both SAC and REDQ equally, and it does not diminish REDQ's advantage in sample efficiency.

The Q-value bias plot, Figure 4 (b), highlights a mechanistic contrast between the two algorithms. SAC exhibits a relatively shallow underestimation bias that fluctuates between -300 and $-1{,}500$ during training. In contrast, REDQ maintains a much deeper underestimation, consistently stabilizing between $-2{,}500$ and $-3{,}000$. This behavior is a direct result of its aggressive update schedule: by performing 20 gradient updates per environment step using a randomized ensemble target, the critics extract more information from each batch. This results in value estimates that are significantly more pessimistic but more stable.

BipedalWalker-v3

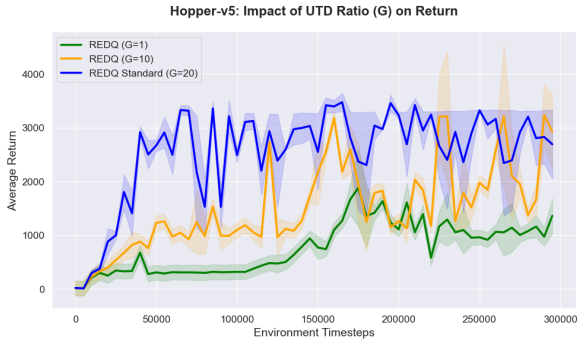
The BipedalWalker-v3 results, Figure 4 (c), provide the most clear demonstration of REDQ's sample efficiency advantage observed in this project. The REDQ Standard agent achieves a return of approximately 290 as early as $\sim$30,000 environment steps and stabilises above 300 from $\sim$60,000 steps onward. On the other hand, the SAC baseline fails to learn any meaningful policy within the 150,000-step, remaining at negative or near-zero returns for the entirety of training, only beginning to trend positive in the final 20,000 steps.

The Q-value bias plot, Figure 4 (d), shows an equally significant divergence. The SAC baseline on BipedalWalker-v3 exhibits a positive bias throughout the entire training, overestimating the true value of states by 50 to 150 points. This overestimation is the direct consequence of SAC's standard Double-Q mechanism proving insufficient for an environment of this complexity. The twin critics correlate and reinforce each other's approximation errors, producing overestimated Q-value targets. The REDQ agent, by contrast, transitions rapidly into a deep, stable underestimation state (reaching -300 by $\sim$50,000 steps), where it remains for the rest of training.

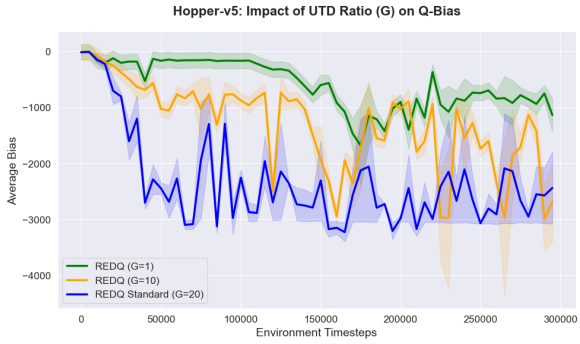
Conclusion

Across both environments, Experiment 1 confirms the primary hypothesis of the REDQ framework. Aggressive UTD ratios accelerate policy learning relative to the SAC baseline. The magnitude of the advantage scales with task complexity: a $6\times$ speedup on Hopper-v5, and a failure of SAC to solve BipedalWalker-v3. The bias analysis provides a mechanistic explanation for this performance gap. On one hand, SAC's failure manifests in diifferent directions depending on the task. It exhibits shallow underestimation in Hopper-v5, but severe overestimation in BipedalWalker-v3. On the other hand, REDQ enforces a stable, controlled underestimation in both environments. This consistency highlights the remarkable robustness of the randomized ensemble mechanism.

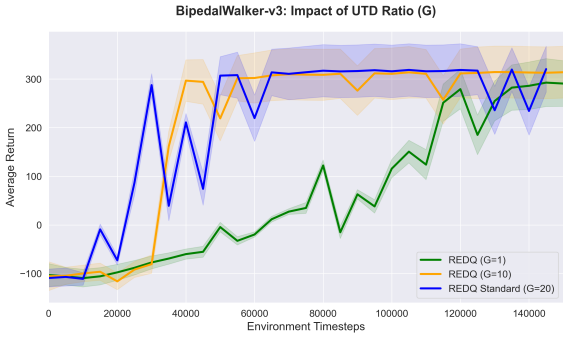
4.2 Experiment 2: The Effect of the UTD Ratio (G)



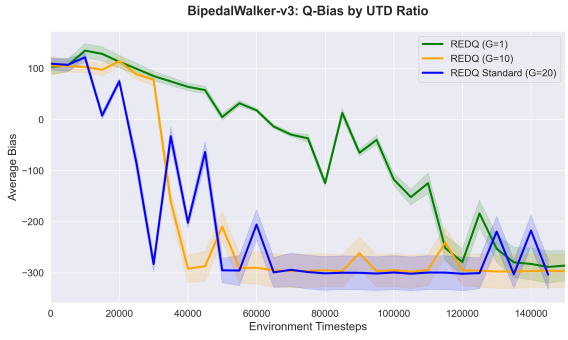
(a) Hopper-v5: Average Return



(b) Hopper-v5: Q-Value Bias



(c) BipedalWalker-v3: Average Return



(d) BipedalWalker-v3: Q-Value Bias

Figure 5: Experiment 2: Ablation study on the UTD ratio on both environments.

Hopper-v5

The UTD ablation on Hopper-v5, Figure 5 (a), provides a clear, monotonic ordering across all three configurations and confirms that the performance gains of REDQ are attributable specifically to the update frequency, rather than to any factor such as ensemble size. The G=20 agent exhibits the most aggressive ramp-up. The G=1 variant, despite sharing the same N=10 ensemble architecture, behaves comparably to SAC. This isolates the UTD ratio as the primary driver of sample efficiency.

The bias curves, Figure 5 (b), reinforce this interpretation with a clear stratification: G=20 induces the deepest underestimation, G=10 sits at an intermediate level, and G=1 remains comparatively shallow. This graduated response demonstrates that bias depth scales proportionally with update frequency. Notably, the G=10 and G=20 curves converge in both return and bias around the 200,000–300,000 step range, suggesting that the primary advantage of G=20 is its dramatically faster initial convergence.

BipedalWalker-v3

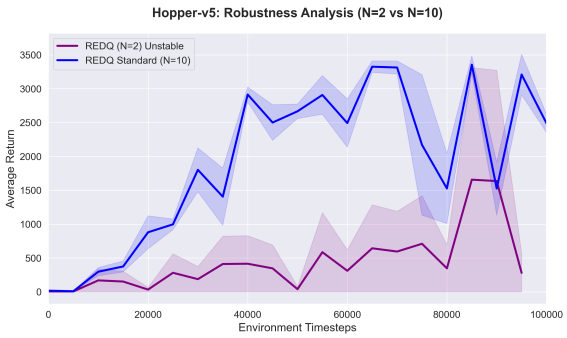
The UTD ablation on BipedalWalker-v3, Figure 5 (c), confirms the previous hierarchy but reveals an important property absent in the noisier Hopper environment. The G=20 agent is the first to break the 300-point threshold, doing so $\sim$20,000 steps ahead of G=10. Both subsequently stabilise near 310, their curves becoming nearly indistinguishable after 60,000 steps. The G=1 variant presents a different trajectory: it begins learning later, only trending upward after 80,000 steps, yet by 140,000 steps it too converges toward the same asymptotic performance of $\sim$300.

The bias curves, Figure 5 (d), corroborate this. The G=1 agent starts with a strong positive bias ($\sim+150$) and only gradually corrects it, eventually settling near -300 alongside the other two configurations. The G=10 and G=20 agents reach this stable state faster, explaining their earlier policy convergence. Crucially, all three agents reach the same final bias level, confirming that the ensemble mechanism successfully constrains overestimation regardless of update frequency.

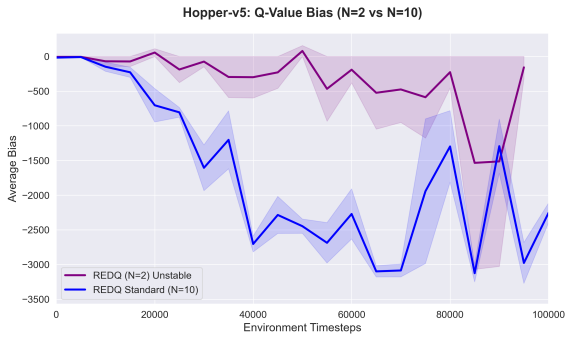
Conclusion

Experiment 2 isolates the UTD ratio as the main driver of REDQ's sample efficiency gains. The two environments show different things: on Hopper-v5, the advantage of G=20 over G=1 is both in speed *and* asymptotic performance. However, on BipedalWalker-v3, all UTD ratios eventually converge to the same value, making the advantage of G=20 purely one of speed. This suggests that the benefit of aggressive update schedules is most pronounced in environments where sample efficiency is the main constraint on final performance.

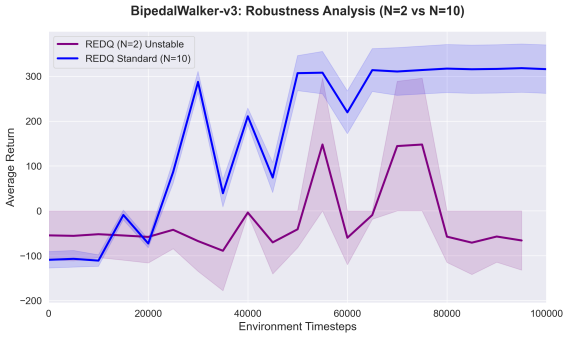
4.3 Experiment 3: Robustness and The Ensemble (N)



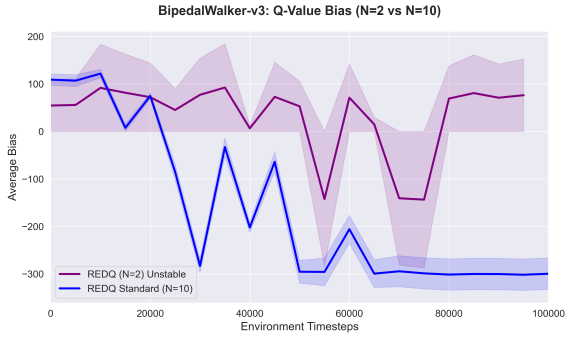
(a) Hopper-v5: Average Return



(b) Hopper-v5: Q-Value Bias



(c) BipedalWalker-v3: Average Return



(d) BipedalWalker-v3: Q-Value Bias

Figure 6: Experiment 3: Ensemble size ablation comparing $N{=}2$ against $N{=}10$ on both environments.

Hopper-v5

Experiment 3 on Hopper-v5, Figure 6 (a), shows one of the clearest results of this project. Having a large ensemble of critics is strictly necessary for stable training when using a high UTD ratio. The $N{=}10$ agent learns quickly and maintains strong performance throughout training, with returns consistently between 2,500 and 3,400. The $N{=}2$ agent starts off similarly, climbing to around 1,600 by $\sim$90,000 steps, but then the return falls to near zero.

Looking at the Q-value bias, Figure 6 (b), explains why this happens. With only two critics, the bias stays very close to zero throughout training, oscillating between 0 and -500. This means the critics are not being conservative enough in their value estimates. They are essentially agreeing with each other and reinforcing their own errors. Without a large enough ensemble to keep the estimates in check, the value predictions gradually become unreliable, which destabilises the policy and leads to the collapse. The $N{=}10$ agent, by contrast, maintains a deep and stable underestimation between $-1{,}500$ and $-3{,}000$ throughout, which is exactly the controlled pessimism that keeps training stable.

BipedalWalker-v3

On BipedalWalker-v3, Figure 6 (c), the result are very similar. Instead of a sudden collapse, the $N{=}2$ agent simply never manages to learn anything useful. The returns oscillate between -100 and $+150$ across the entire 100,000-step run and never stabilise.

The bias plot, Figure 6 (d), shows the problem. The $N{=}2$ agent has a positive bias during training, meaning it is consistently overestimating how good its actions are. This is the overestimation problem that REDQ is specifically designed to prevent. With only two critics running ten gradient updates per step, they quickly start overfitting and produce overestimated values that mislead the policy. Unlike in Hopper, the environment here does not cause a single dramatic crash, but the overestimation is persistent enough to prevent any learning. The $N{=}10$ agent, by contrast, quickly corrects its initial overestimation, reaches a stable underestimation around -300 by 50,000 steps, and achieves near-optimal performance for the rest of training.

Conclusion

Experiment 3 makes it clear that a large critic ensemble is not optional. Without it, running a high UTD ratio leads to failure, either as a sudden collapse as in Hopper-v5, or as a persistent inability to learn as in BipedalWalker-v3. In both cases the root cause is the same: with too few critics, the value estimates become unreliable and the policy degrades.

5 Reproducibility and Critical Commentary

5.1 Hardware Bottlenecks and Wall-Clock Time

While REDQ is remarkably sample-efficient, practical implementation revealed a severe computational trade-off. Because the algorithm executes G gradient updates for every single environment step, the workload shifts heavily from environment simulation to neural network backpropagation, causing a high increase in wall-clock time as G scales. As shown in Table 1, a run that completes in under an hour with the SAC baseline ($G = 1$) escalates to nearly 13–27 hours per seed with REDQ, depending on the environment and UTD ratio.

Environment	Algorithm	UTD (G)	Timesteps	Avg. Wall-Clock Time (per seed)
Hopper-v5	SAC Baseline	$G = 1$	300,000	45.20 ± 0.58 minutes
Hopper-v5	REDQ Intermediate	$G = 10$	300,000	12.72 ± 0.09 hours
Hopper-v5	REDQ Standard	$G = 20$	300,000	26.93 ± 0.03 hours
BipedalWalker-v3	SAC Baseline	$G = 1$	150,000	48.37 ± 0.32 minutes
BipedalWalker-v3	REDQ Intermediate	$G = 10$	150,000	9.03 ± 0.09 hours
BipedalWalker-v3	REDQ Standard	$G = 20$	150,000	13.25 ± 0.15 hours

Table 1: Comparison of physical execution times per seed. Different timesteps on different environments.

This bottleneck imposed two practical limitations. First, only 2 seeds were evaluated per configuration instead of the 5 or more typically required for statistical robustness, so the variance estimates in the figures should be interpreted with caution. Second, for Experiment 3, where the goal was simply to confirm whether the ensemble size matters, not to study long-term dynamics, the N=2 configuration was run for only 100,000 steps on both environments. Since the instability of the N=2 agent is evident well before this point in both cases, this decision does not affect the validity of the conclusions.

5.2 Overall Assessment

The implementation of REDQ was successful, and the empirical results perfectly align with the results of the original paper. The algorithm effectively solves the sample inefficiency of model-free RL. However, the practical experience of implementing it highlighted that REDQ trades sample complexity for computational complexity.

6 Declaration of AI Usage

During the preparation of this report, AI tools were used exclusively for the purposes of revising phrasing, grammar and spelling to improve the overall readability of the text. No AI tools were used to generate the source code or analyze the experimental results. All algorithmic implementations, empirical experiments, and final conclusions presented in this work are entirely my own original work.

References

[1] Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross. Randomized ensembled double q-learning: Learning fast without a model, 2021.

[2] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods, 2018.

[3] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024.